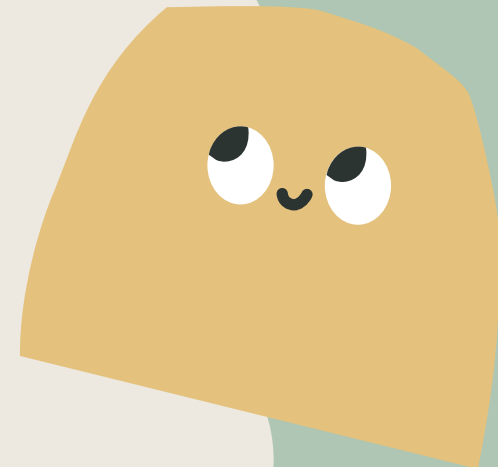
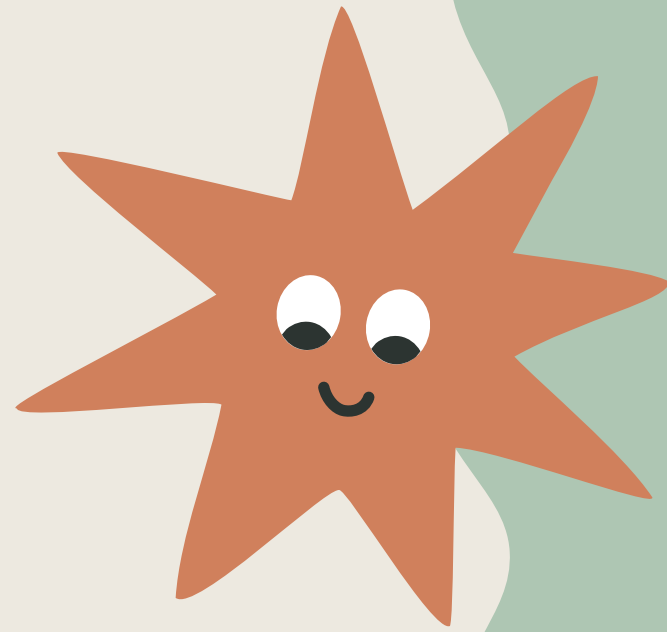


Stack Implementation Using Linked List in C++

Lecture 14



Learning Outcomes (Lecture 13)

➤ At the end of this lecture, students will be able to:

- Implement a Stack using a singly linked list
- Create Node and Stack classes
- Understand how TOP is represented using a pointer
- Implement
 - push(), pop(), peek(), isEmpty(), display()
- Handle Stack Overflow and Underflow
- Understand why Stack insertion/deletion should occur at the beginning of the linked list
- Analyze the time complexity of each operation

HOW A LINKED LIST REPRESENTS A STACK?

A linked list can be used to implement a Stack by performing all insertions and deletions at the **beginning (head)** of the list.

CONCEPT

- **TOP** is a pointer that points to the first node of the linked list.
- The first node is always the top of the stack.
- **Push** adds a new node at the beginning.
- **Pop** removes the first node.

STACK USING LINKED LIST

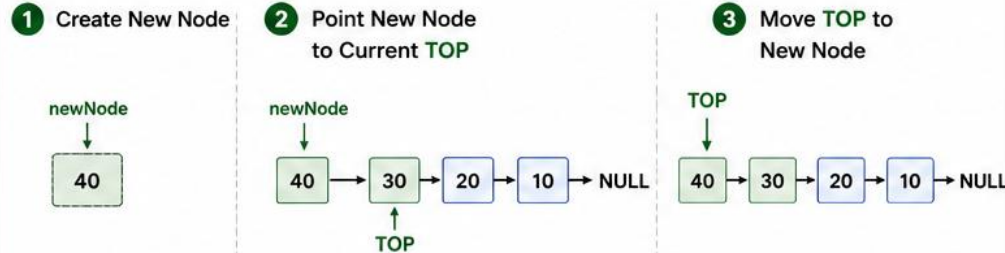


Here, **TOP** points to the first node (30).
So, **30** is the top element of the stack.

WHY AT THE BEGINNING?

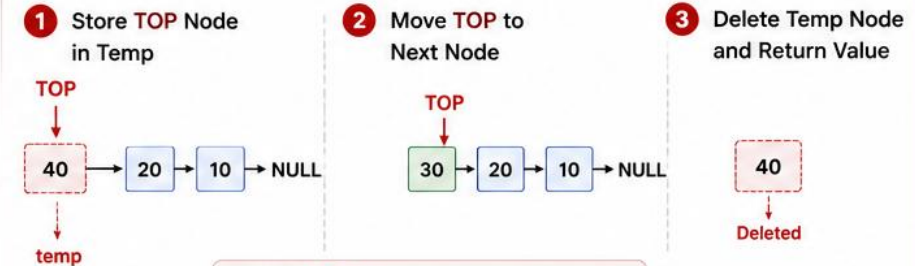
- Inserting at the beginning takes $O(1)$ time.
- Deleting from the beginning takes $O(1)$ time.
- If we insert/delete at the end, we may need to traverse the list which takes $O(n)$ time.

PUSH OPERATION (Insert at Beginning)



After **Push(40)**: **TOP** points to 40. New node becomes the top of the stack.

POP OPERATION (Delete from Beginning)

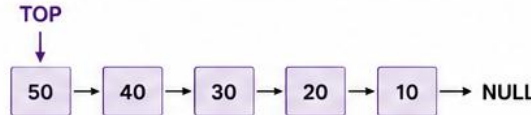


After **Pop()**: node 40 is removed.
TOP now points to 30.

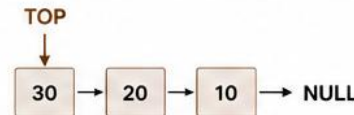
INITIAL STATE (Empty Stack)



STACK AFTER SEVERAL PUSH OPERATIONS



STACK AFTER SEVERAL POP OPERATIONS



SUMMARY

- ✓ **TOP** is the head of the linked list.
- ✓ **Push** = Insert at beginning.
- ✓ **Pop** = Delete from beginning.
- ✓ All main stack operations take $O(1)$ time.



Key Point: In a linked list based stack, we always work at the **beginning (head)** of the list to maintain $O(1)$ time for **Push** and **Pop**.

isEmpty()

Algorithm:

1. If TOP == NULL
 return TRUE
2. Otherwise
 return FALSE

C++:

```
bool isEmpty()  
{  
    return top == NULL;  
}
```

Complexity

O(1)

push()

Algorithm:

1. Create a new node.
2. Store value in the node.
3. Make newNode->next point to TOP.
4. Move TOP to the new node.
5. Stop.

C++:

```
void push(int value)
{
    Node* newNode = new Node(value);

    newNode->next = top;
    top = newNode;
}
```

Complexity

O(1)

pop()

Algorithm:

1. Check whether Stack is empty.
2. If empty:
 Underflow.
 Stop.
3. Store TOP in temp.
4. Move TOP to TOP->next.
5. Delete temp.
6. Return removed value.

C++:

```
int pop()
{
    if (isEmpty())
    {
        cout << "Stack Underflow!";
        return -1;
    }

    Node* temp = top;
    int value = temp->data;

    top = top->next;

    delete temp;

    return value;
}
```

Complexity

O(1)

peek()

Algorithm:

1. Check if Stack is empty.
2. If empty:
Underflow.
3. Otherwise:
Return arr[TOP].
4. Stop.

C++:

```
int peek()
{
    if (isEmpty())
    {
        cout << "Stack is empty!";
        return -1;
    }

    return top->data;
}
```

Complexity

O(1)

display()

Algorithm:

1. Set current = TOP.
2. While current != NULL:
 Display current->data.
 Move current to current->next
3. Stop.

C++:

```
void display()
{
    if (isEmpty())
    {
        cout << "Stack is empty!";
        return;
    }

    Node* current = top;

    cout << "Stack:\n";

    while (current != NULL)
    {
        cout << current->data << endl;
        current = current->next;
    }
}
```

Complexity

O(n)